VISIÓN ARTIFICIAL · LIC. EN INTELIGENCIA ARTIFICIAL · 7° SEMESTRE

Práctica 2 · Adquisición de video y filtrado

Capturar video de la webcam, medir lo que entrega la cámara, ensuciar la imagen con ruido y limpiarla con filtros

Modalidad	Individual, en clase y junto con el docente (programación en vivo)
Duración	2 horas; lo que no se termine en clase se termina de tarea
Temas	2.1–2.3 Adquisición y video · 3.1 Filtrado · 3.2 Eliminación del ruido y realce
Herramientas	Python 3.10+ · OpenCV · NumPy · scikit-image (para la Práctica 3) · webcam
Entregable	Los 3 scripts + reporte breve con capturas y respuestas (sección 8)

Objetivo

Que escribas los scripts que **leen video en tiempo real** (de la webcam o de un archivo), midan qué entrega realmente la cámara, y apliquen sobre cada frame los **filtros de suavizado y realce** que se usan como preprocesamiento antes de segmentar.

Cómo está organizada la práctica

Igual que en la Práctica 1: **cada paso es un script nuevo que parte del anterior**. Esta vez la parte común (`[BASE]`) es el **ciclo de video**, y la vas a reutilizar sin cambios también en la Práctica 3.

Paso	Script	Qué agrega	Qué vas a ver
1	`p2_paso1_video.py`	Abrir la cámara o un video, propiedades, FPS, foto y grabación	El video con FPS y resolución encima; un `.mp4` grabado
2	`p2_paso2_suavizado.py`	Kernel y convolución, ruido, 4 filtros, PSNR y tiempo	Un mosaico con 6 versiones del video en vivo
3	`p2_paso3_realce.py`	Contraste (ecualización, CLAHE) y nitidez	Imágenes y sus histogramas en vivo

Los resultados se muestran en **ventanas de OpenCV**. Para cambiar parámetros en vivo usamos **barras deslizantes** (*trackbars*) de OpenCV: no es una interfaz, son sliders dentro de la misma ventana.

Contenido

- Objetivo
- Cómo está organizada la práctica

- 1. Preparar el entorno
- 2. Conceptos clave
- 3. Paso 1 · Adquisición de video
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 1
- 4. Paso 2 · Convolución, ruido y suavizado
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 2
- 5. Paso 3 · Realce: contraste y nitidez
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 3
- 6. Retos (opcionales, suman puntos)
- 7. Errores frecuentes
- 8. Entregable
  - Referencia rápida

1. Preparar el entorno

Usa la misma carpeta de trabajo y el mismo entorno virtual `va` de la Práctica 1 (o crea uno nuevo igual que en la sección 1.3 de la P1). Activa el entorno y actualiza las librerías con el nuevo `requirements.txt`:

```
opencv-python>=4.8
numpy>=1.24
matplotlib>=3.7
scikit-image>=0.22
```

```
pip install -r requirements.txt
```

> **¿Por qué scikit-image si no se usa aquí?** La Práctica 3 la necesita para los contornos activos. Es una descarga grande (≈ 70 MB con sus dependencias), así que conviene instalarla hoy y no a mitad de la siguiente práctica.

Estructura de la carpeta:

```
practica02/
├── recursos/
│   └── video_sintetico.mp4     <- lo comparte el docente (para quien no tenga cámara)
├── requirements.txt
├── p2_paso1_video.py
├── p2_paso2_suavizado.py
└── p2_paso3_realce.py
```

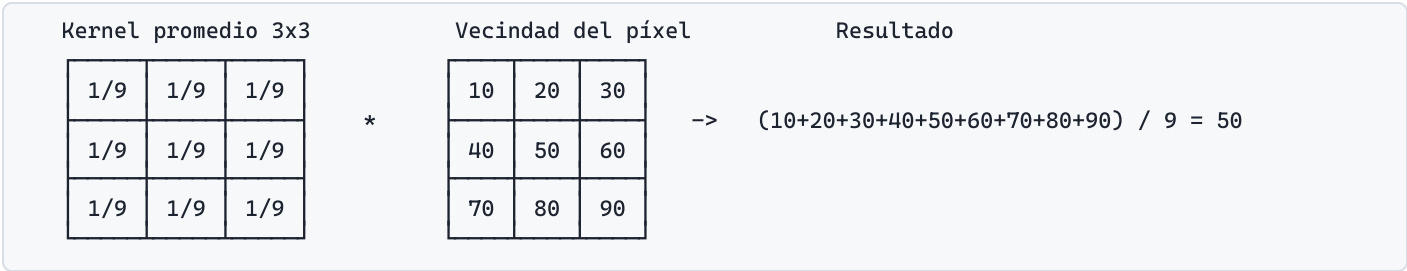
Cada script funciona de dos formas:

```
python p2_paso1_video.py            # webcam
python p2_paso1_video.py --video    # recursos/video_sintetico.mp4 (o: --video otra_ruta.mp4)
```

El **video sintético** es una escena de 10 s, 640×360 y 30 FPS: un fondo gris con iluminación dispareja, tres líneas rectas y figuras de colores en movimiento. Se repite en bucle. Las capturas de este documento están hechas con él.

2. Conceptos clave

- **Video:** una secuencia de imágenes (*frames*). Procesar video es repetir, para cada frame: **leer** → **procesar** → **mostrar** → **revisar el teclado**.
- **FPS (frames por segundo):** lo que la cámara **dice** entregar (`CAP_PROP_FPS`) no siempre es lo que **logras**: la exposición, el USB, el formato y tu propio procesamiento lo reducen. Por eso se **mide**.
- **`cv2.waitKey(ms)`:** espera unos milisegundos, deja que la ventana se redibuje y devuelve la tecla pulsada. Sin él, las ventanas no se actualizan.
- **Kernel:** una matriz pequeña (3×3, 5×5…) de pesos. La **convolución** recorre la imagen y reemplaza cada píxel por la suma de sus vecinos multiplicados por el kernel.
- **Ruido:** variaciones aleatorias que no son de la escena. El **gaussiano** (sensor) afecta un poco a todos los píxeles; el de **sal y pimienta** (transmisión) afecta mucho a unos cuantos.
- **PSNR (dB):** qué tan parecida es una imagen a la original. Más alto = más parecida. Por encima de 30 dB la diferencia apenas se nota.
- **Histograma:** cuántos píxeles hay de cada nivel de gris. Una imagen con poco contraste tiene el histograma **apretado** en una zona.



3. Paso 1 · Adquisición de video

Lo que se verá

Cómo abrir una cámara o un archivo con `cv2.VideoCapture`, qué propiedades reporta (resolución, FPS, códec), cómo **pedir** otra resolución y comprobar si la cámara la aceptó, cómo medir los FPS reales y cómo guardar una foto y grabar video con `cv2.VideoWriter`.

Qué tienes que hacer

Crea `p2_paso1_video.py`.

1. Sección `[BASE]` de video. Escríbela completa: la usarás en los pasos 2 y 3 y en toda la Práctica 3.

```python
import sys                          # sys.argv (argumentos) y sys.exit (terminar con un mensaje)
import time                         # time.perf_counter(): reloj de alta precisión para medir FPS y ms
from pathlib import Path            # rutas que funcionan igual en Windows, macOS y Linux

import cv2                          # OpenCV: cámara, video, filtros, segmentación y ventanas
import numpy as np                  # NumPy: cada frame es un arreglo de NumPy

# ============================================================================
# [BASE] Video: abrir la fuente, leer frames, medir FPS, escribir texto, mosaico y barras
# ============================================================================
CARPETA = Path(__file__).resolve().parent                    # carpeta donde está este script
RUTA_VIDEO = CARPETA / "recursos" / "video_sintetico.mp4"    # video de respaldo (sin cámara)
SALIDAS = CARPETA / "salidas"                                # aquí se guardan capturas y grabaciones


def abrir_fuente():
    """Webcam por defecto; con --video usa RUTA_VIDEO (o la ruta que siga a --video).
    Devuelve (captura, es_archivo, espera_ms)."""
    if "--video" in sys.argv:                                # ¿se pidió un archivo de video?
        i = sys.argv.index("--video")                        # posición de "--video" en la lista
        ruta = sys.argv[i + 1] if i + 1 < len(sys.argv) else RUTA_VIDEO   # ruta escrita o la de respaldo
        captura = cv2.VideoCapture(str(ruta))                # abrir el archivo
        es_archivo = True
    else:
        captura = cv2.VideoCapture(0)                        # 0 = primera cámara del equipo
        es_archivo = False
    if not captura.isOpened():                               # no existe, está ocupada o el códec falla
        sys.exit("No se pudo abrir la fuente de video (¿cámara ocupada? prueba con --video)")
    fps = captura.get(cv2.CAP_PROP_FPS) or 30                # algunos drivers reportan 0 -> usamos 30
    espera = max(1, int(1000 / fps)) if es_archivo else 1    # un archivo se reproduce a su velocidad real
    return captura, es_archivo, espera


def leer_frame(captura, es_archivo):
    """Lee el siguiente frame. Un archivo vuelve a empezar al terminar. Devuelve None si no hay imagen."""
    ok, frame = captura.read()                               # ok = True si llegó un frame
    if not ok and es_archivo:                                # fin del archivo: regresamos al frame 0
        captura.set(cv2.CAP_PROP_POS_FRAMES, 0)
```

```
        ok, frame = captura.read()
    return frame if ok else None


class MedidorFPS:
    """Mide los cuadros por segundo reales (promedio suavizado)."""

    def __init__(self):
        self.anterior = time.perf_counter()                  # instante del frame anterior
        self.fps = 0.0

    def actualizar(self):
        ahora = time.perf_counter()
        dt = ahora - self.anterior                           # segundos entre este frame y el anterior
        self.anterior = ahora
        if dt > 0:
            self.fps = 1 / dt if self.fps == 0 else 0.9 * self.fps + 0.1 / dt   # suaviza los saltos
        return self.fps


def escribir(img, texto, fila=0, color=(255, 255, 255)):
    """Escribe texto en la esquina superior izquierda, sobre un rectángulo negro para que se lea siempre."""
    y = 22 + 24 * fila                                       # cada fila de texto baja 24 px
    (w, h), base = cv2.getTextSize(texto, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)   # tamaño que ocupará el texto
    cv2.rectangle(img, (4, y - h - 5), (12 + w, y + base + 2), (0, 0, 0), -1)   # -1 = rectángulo relleno
    cv2.putText(img, texto, (8, y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1, cv2.LINE_AA)


def mosaico(imagenes, titulos, columnas=3, ancho=400):
    """Une varias imágenes (color o gris) en una sola, en una rejilla con título en cada celda."""
    celdas = []
    for img, titulo in zip(imagenes, titulos):
        if img.ndim == 2:                                    # gris o máscara -> 3 canales para poder unirla
            img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
        alto = int(img.shape[0] * ancho / img.shape[1])      # conservar la proporción
        celda = cv2.resize(img, (ancho, alto))               # todas las celdas del mismo tamaño
        escribir(celda, titulo, color=(0, 255, 255))         # título en amarillo
        celdas.append(celda)
    while len(celdas) % columnas:                            # completar la última fila con celdas negras
        celdas.append(np.zeros_like(celdas[-1]))
    filas = [np.hstack(celdas[i:i + columnas]) for i in range(0, len(celdas), columnas)]   # pegar en horizontal
    return np.vstack(filas)                                  # y luego las filas en vertical


def barra(nombre, ventana, inicial, maximo):
    """Crea una barra deslizante (trackbar) en una ventana que ya existe."""
    cv2.createTrackbar(nombre, ventana, inicial, maximo, lambda v: None)   # lambda: no hacer nada al moverla


def valor(nombre, ventana):
    """Lee el valor actual de una barra deslizante."""
    return cv2.getTrackbarPos(nombre, ventana)
```

Lo importante de la base:

- `abrir_fuente()` decide entre webcam y archivo según `--video`, y calcula cuánto esperar entre frames: un archivo debe reproducirse a su velocidad (33 ms a 30 FPS); la cámara ya llega a su ritmo.
- `leer_frame()` devuelve `None` si la cámara falla y, si es un archivo, vuelve a empezar al terminar.
- `MedidorFPS` mide el tiempo entre frames con `time.perf_counter()`.
- `escribir()` pone texto legible sobre el video; `mosaico()` une varias imágenes en una sola ventana con títulos (la usarás desde el paso 2).
- `barra()` y `valor()` crean y leen las barras deslizantes.

2. Programa principal. Debajo de la base:

a. Abre la fuente e imprime sus propiedades:

```
captura, es_archivo, espera = abrir_fuente()
print("backend    :", captura.getBackendName())
print("resolución :", int(captura.get(cv2.CAP_PROP_FRAME_WIDTH)), "x",
        int(captura.get(cv2.CAP_PROP_FRAME_HEIGHT)))
print("FPS        :", captura.get(cv2.CAP_PROP_FPS))
```

b. Si es cámara, **pide** 1280×720 con `captura.set(...)` y vuelve a leer con `captura.get(...)` lo que realmente quedó.

c. El ciclo de video: leer el frame, medir FPS, dibujar el texto sobre una **copia** y mostrar:

```
medidor = MedidorFPS()
while True:
    frame = leer_frame(captura, es_archivo)
    if frame is None:
        break
    fps = medidor.actualizar()
    vista = frame.copy()
    escribir(vista, f"{frame.shape[1]}x{frame.shape[0]}  |  {fps:5.1f} FPS medidos")
    cv2.imshow("Paso 1 - Video", vista)
    tecla = cv2.waitKey(espera) & 0xFF
    if tecla in (27, ord("q")):
        break
```

d. Teclas: con **S** guarda el frame con `cv2.imwrite`; con **R** crea un `cv2.VideoWriter` (o lo cierra si ya existía). Mientras grabas, escribe cada frame **limpio** con `escritor.write(frame)` y dibuja un punto rojo en la vista:

```
codigo = cv2.VideoWriter_fourcc(*"mp4v")
escritor = cv2.VideoWriter("salidas/grabacion.mp4", codigo, 30, (ancho, alto))   # tamaño = (ANCHO,
        ALTO)
```

e. Al salir: `escritor.release()`, `captura.release()`, `cv2.destroyAllWindows()` y un resumen con los frames leídos y los FPS promedio.

Resultado esperado

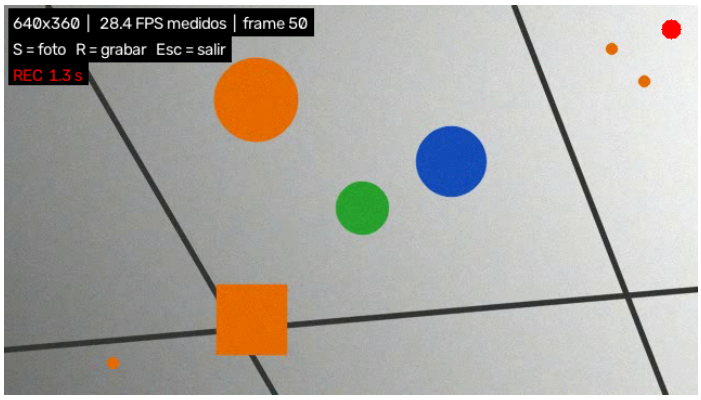
Con `--video`, en la consola (con webcam verás tu backend, por ejemplo `MSMF` o `DSHOW`, y la línea de la resolución pedida):

```
== 1. Propiedades de la fuente ==
tipo       : archivo
backend    : FFMPEG
resolución : 640 x 360
```

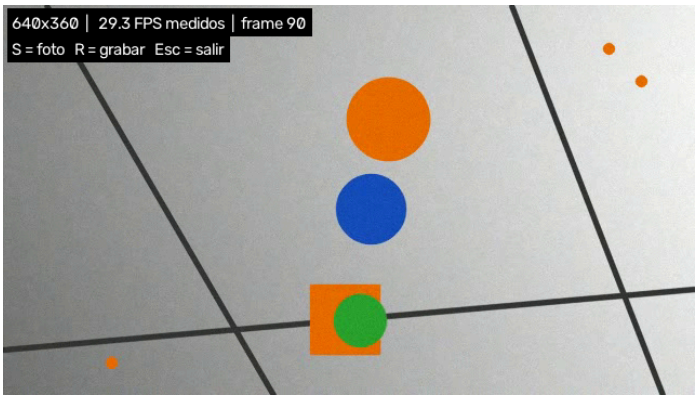
```
FPS        : 30.0
códec      : FMP4
frames     : 300

S = guardar frame · R = grabar / detener · Esc o Q = salir
grabando en: grabacion_232305.mp4
guardado: frame_00040.png
grabación terminada: 60 frames (2.0 s a 30 FPS)

== 4. Resumen ==
frames leídos: 90   tiempo: 3.2 s   FPS promedio: 28.0
```



Grabando: punto rojo y contador de segundos



Sin grabar

En la carpeta `salidas/` quedan el PNG y el `.mp4` grabado.

Preguntas del paso 1

1. Con tu webcam: ¿cuántos FPS **reporta** y cuántos **mides**? Tapa la cámara con la mano o apaga la luz: ¿qué pasa con los FPS medidos? ¿Por qué?
2. Pediste 1280×720. ¿Qué resolución obtuviste? ¿Por qué `set()` no garantiza nada?
3. Graba 10 s **reales** (cuéntalos con un reloj). ¿Cuánto dura el archivo al reproducirlo? Si no coincide, explica por qué tiene que ver con el `30` que pasamos a `VideoWriter`.
4. ¿Por qué se graba `frame` y no `vista`?
5. ¿Qué pasa si cambias `cv2.waitKey(espera)` por `cv2.waitKey(0)` dentro del ciclo? ¿Y si lo quitas?

4. Paso 2 · Convolución, ruido y suavizado

Lo que se verá

Un **filtro de suavizado** reemplaza cada píxel por una combinación de sus vecinos, con lo que se reduce el ruido a cambio de perder detalle. Compararemos cuatro, sobre el video en vivo y con ruido artificial controlado:

Filtro	Cómo combina la vecindad	Bueno para	Problema
Promedio	Todos los vecinos pesan igual (kernel de $1/k^2$)	Ruido gaussiano leve	Borra los bordes
Gaussiano	Pesos con forma de campana: el centro pesa más	Ruido gaussiano	También suaviza bordes, aunque menos
Mediana	Ordena los vecinos y toma el de en medio (no es convolución)	**Sal y pimienta**	Más lento; redondea esquinas
Bilateral	Promedia solo vecinos **cercanos y de color parecido**	Quitar ruido **conservando bordes**	El más lento

Qué tienes que hacer

1. **Copia** `p2_paso1_video.py` como `p2_paso2_suavizado.py`, conserva la `[BASE]` y borra el programa principal del paso 1.
2. **Agrega** las funciones de ruido y la de medir tiempo:

```
108 def ruido_gaussiano(img, sigma):
109     """Suma a cada píxel un número aleatorio con media 0 y desviación sigma (ruido del sensor)."""
110     ruido = np.random.normal(0, sigma, img.shape)             # matriz de números aleatorios (float)
111     return np.clip(img + ruido, 0, 255).astype(np.uint8)      # clip: recorta a 0..255 antes de
    volver a uint8
112
113
114 def ruido_sal_pimienta(img, proporcion):
115     """Pone en blanco (sal) o negro (pimienta) una proporción de píxeles al azar (fallas de
    transmisión)."""
116     salida = img.copy()
117     azar = np.random.rand(*img.shape[:2])                     # un número 0..1 por píxel
118     salida[azar < proporcion / 2] = 0                         # pimienta
119     salida[azar > 1 - proporcion / 2] = 255                   # sal
120     return salida
121
122
123 def medir(funcion, *argumentos):
124     """Ejecuta funcion(*argumentos) y devuelve (resultado, milisegundos que tardó)."""
125     t0 = time.perf_counter()
126     resultado = funcion(*argumentos)
127     return resultado, (time.perf_counter() - t0) * 1000
```

3. **Convolución a mano.** Antes del ciclo, construye el kernel promedio de 5×5, aplícalo con `cv2.filter2D` y comprueba que da lo mismo que `cv2.blur`:

```
k = 5
kernel = np.ones((k, k), np.float32) / (k * k)
a_mano = cv2.filter2D(frame, -1, kernel)
print("diferencia:", cv2.absdiff(a_mano, cv2.blur(frame, (k, k))).max())
```

4. **Barras.** Crea la ventana y tres barras: `Kernel` (k = 2·valor + 1), `Ruido` (0 ninguno, 1 gaussiano, 2 sal y pimienta) e `Intensidad`:

```
VENTANA = "Paso 2 - Suavizado"
cv2.namedWindow(VENTANA)                # la ventana debe existir antes de crear barras
barra("Kernel", VENTANA, 2, 7)
barra("Ruido", VENTANA, 1, 2)
barra("Intensidad", VENTANA, 25, 100)
```

5. **En el ciclo:** lee las barras, ensucia el frame según el tipo de ruido y aplica los 4 filtros midiendo su tiempo:

```
promedio, ms_p = medir(cv2.filter2D, ruidoso, -1, kernel)
gauss, ms_g = medir(cv2.GaussianBlur, ruidoso, (k, k), 0)
mediana, ms_m = medir(cv2.medianBlur, ruidoso, k)
bilateral, ms_b = medir(cv2.bilateralFilter, ruidoso, k, 75, 75)
```

6. **Muestra un mosaico** de 2×3 (original, ruidoso y los 4 filtros) con el **PSNR contra el frame limpio** y los ms en cada título: `cv2.PSNR(frame, gauss)`.
7. Al salir, imprime una tabla con el PSNR y el tiempo de cada filtro en el último frame.

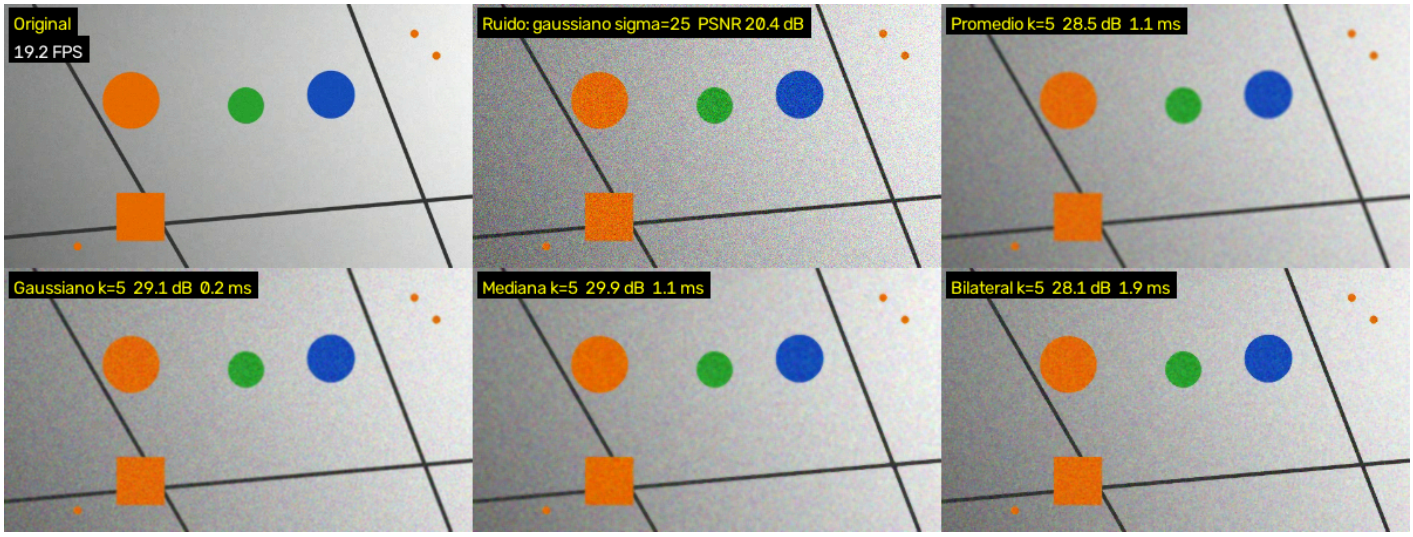
Resultado esperado

```
== 1. Kernel promedio 5x5 ==
[[0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]]
diferencia máxima filter2D vs cv2.blur: 0

Mueve las barras. Esc o Q = salir

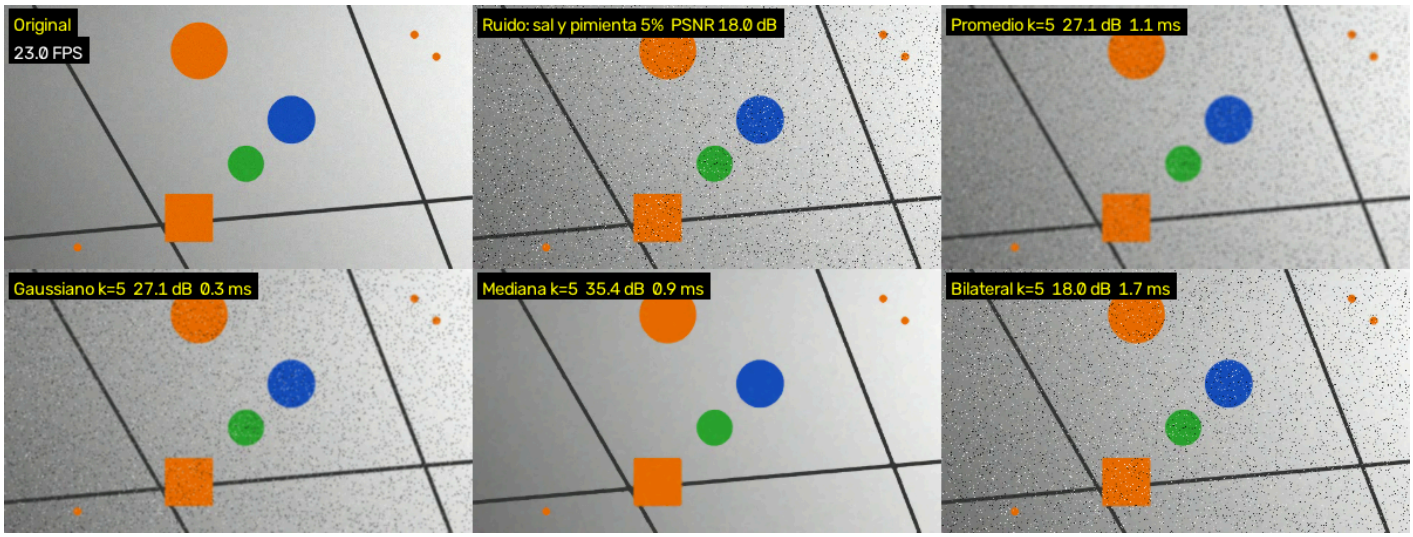
== 6. Último frame: ruido sal y pimienta 5%, kernel 5x5 ==
filtro     PSNR (dB)  tiempo (ms)
ruidoso         18.0            -
Promedio        27.1         1.09
Gaussiano       27.1         0.31
Mediana         35.4         0.91
Bilateral       18.0         1.73
```

Ruido **gaussiano** (σ = 25): los cuatro filtros mejoran el PSNR y el gaussiano y la mediana quedan arriba.



Ruido gaussiano, kernel 5×5

Ruido **sal y pimienta** (5 %): la **mediana** lo elimina casi por completo (35 dB); el promedio y el gaussiano solo lo "embarran" y el bilateral casi no hace nada.



Sal y pimienta, kernel 5×5

> Con la barra `Ruido` en 0, el PSNR del panel "Ruido" marca **361 dB**: es el valor que OpenCV devuelve cuando las dos imágenes son idénticas (en teoría, infinito).

Preguntas del paso 2

1. Llena una tabla con el PSNR de cada filtro para ruido gaussiano y para sal y pimienta (k = 5). ¿Cuál gana en cada caso? ¿Por qué la mediana es tan buena con sal y pimienta?
2. Con ruido gaussiano, sube el kernel de 3 a 15. ¿El PSNR siempre mejora? ¿Qué se pierde?
3. ¿Qué filtro es el más lento? ¿Cómo hace el bilateral para no borrar los bordes?
4. Los pesos del kernel promedio suman 1. ¿Qué le pasaría a la imagen si sumaran 2? ¿Y 0.5?
5. `filter2D` y `cv2.blur` dieron diferencia 0. ¿Qué demuestra eso sobre lo que hace `cv2.blur`?

5. Paso 3 · Realce: contraste y nitidez

Lo que se verá

- **Contraste.** Simulamos una cámara en un lugar mal iluminado bajando el contraste (`alfa·gris + beta`). Lo recuperamos de dos formas:
  - **Ecualización del histograma:** reparte los niveles en todo el rango 0–255. Es global: una sola regla para toda la imagen.
  - **CLAHE:** ecualiza por zonas (una rejilla de 8×8) y **limita** cuánto puede crecer el contraste (*clip*) para no amplificar el ruido.
- **Nitidez.** Lo contrario de suavizar: realzar los bordes.
  - **Kernel de nitidez:** `[[0,-1,0],[-1,5,-1],[0,-1,0]]`, el centro menos sus 4 vecinos.
  - **Unsharp mask:** `original + f·(original − desenfocada)`. Le suma a la imagen sus propios detalles.

Qué tienes que hacer

1. **Copia** el paso 1 como `p2_paso3_realce.py`, conserva la `[BASE]` y borra el programa principal.
2. **Agrega** la función que dibuja un histograma como imagen:

```
def histograma(gris, alto=150):
    """Dibuja el histograma de una imagen gris como una imagen de 256 x alto píxeles."""
    h = cv2.calcHist([gris], [0], None, [256], [0, 256]).ravel()     # cuántos píxeles tienen cada valor 0..255
    h = h / (h.max() + 1e-9) * (alto - 5)                            # escalar al alto del dibujo
    lienzo = np.full((alto, 256), 255, np.uint8)                     # fondo blanco
    for x, y in enumerate(h):
        cv2.line(lienzo, (x, alto), (x, alto - int(y)), 0, 1)        # una barra vertical por nivel de gris
    return lienzo
```

3. Crea la ventana con 4 barras: `Contraste %` (35), `Brillo` (40), `Clip CLAHE` (40 → 4.0) y `Afilado` (15 → 1.5).
4. **En el ciclo**, con el frame en gris:

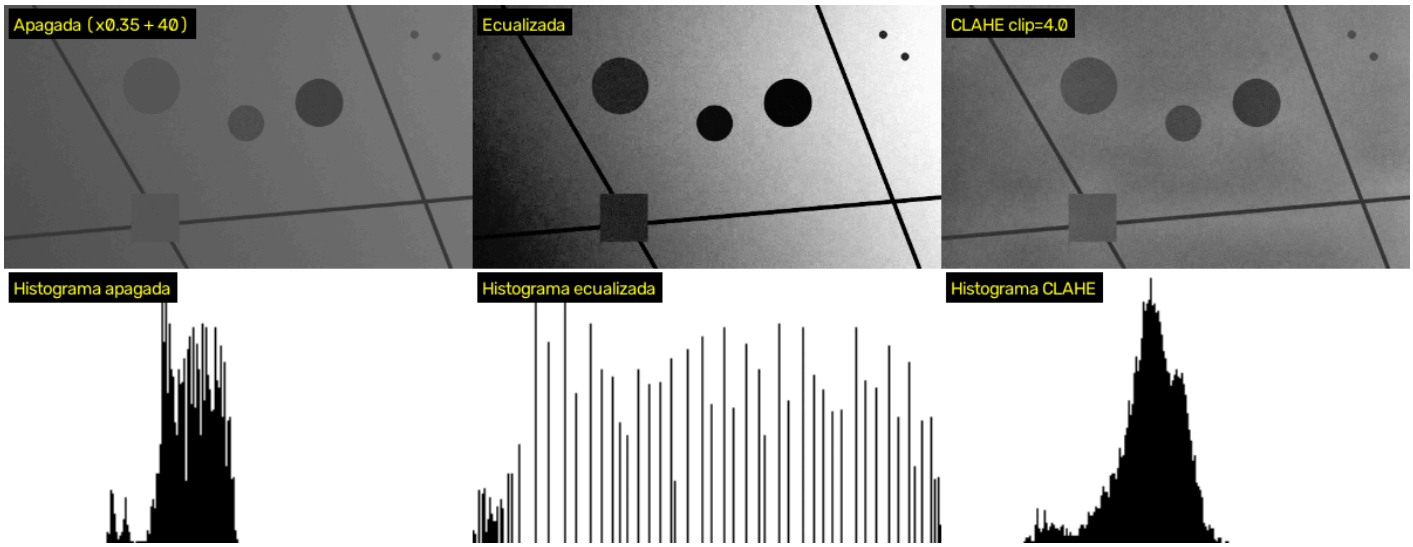
```
apagada = cv2.convertScaleAbs(gris, alpha=alfa, beta=beta)          # mala iluminación
ecualizada = cv2.equalizeHist(apagada)
clahe = cv2.createCLAHE(clipLimit=clip, tileGridSize=(8, 8))
local = clahe.apply(apagada)
con_kernel = cv2.filter2D(gris, -1, nitidez)                        # nitidez = el kernel 3x3
desenfocada = cv2.GaussianBlur(gris, (0, 0), 3)
unsharp = cv2.addWeighted(gris, 1 + fuerza, desenfocada, -fuerza, 0)
```

5. Muestra dos ventanas: **contraste** (arriba apagada, ecualizada y CLAHE; abajo sus tres histogramas) y **nitidez** (original, kernel y unsharp).

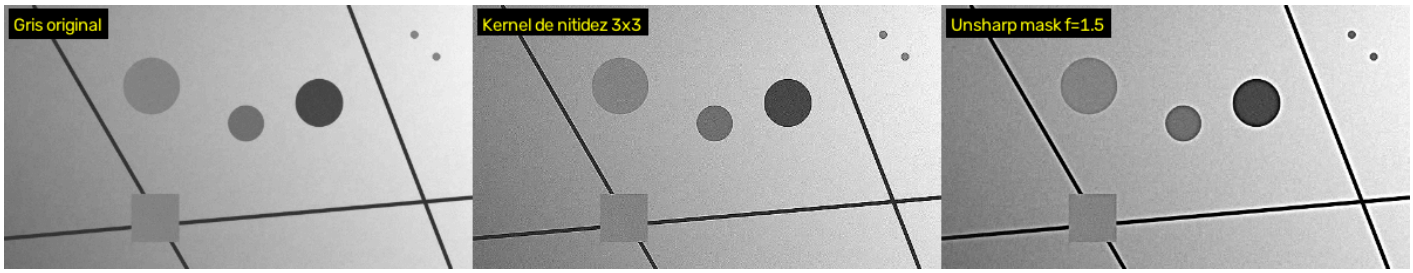
Resultado esperado

```
== Último frame ==
original    min= 40  max=253  desviación estándar= 41.3
apagada     min= 54  max=129  desviación estándar= 14.5
```

```
ecualizada  min=  0  max=255  desviación estándar= 73.6
CLAHE       min= 42  max=161  desviación estándar= 18.5
```



Contraste: la imagen apagada tiene el histograma apretado; la ecualización lo estira a todo el rango; CLAHE lo abre con moderación



Nitidez: el kernel 3×3 y el unsharp mask marcan los bordes; con fuerza alta aparecen halos claros junto a los bordes oscuros

Preguntas del paso 3

1. Compara los tres histogramas. ¿Qué hizo la ecualización con los niveles? ¿Por qué el histograma ecualizado tiene “huecos” entre barras?
2. La ecualización global hace muy visible el ruido del fondo. ¿Qué hace distinto CLAHE para evitarlo? Sube el *clip* a 10: ¿qué pasa?
3. Sube el afilado a 5. Describe los halos que aparecen. ¿Por qué el afilado también hace más visible el ruido?
4. Los pesos del kernel de nitidez suman 1. ¿Qué le pasa a una zona plana (todos los píxeles iguales) al aplicarlo? ¿Y a un borde?
5. En un sistema de visión, ¿en qué caso pondrías una ecualización **antes** de umbralizar? ¿Y en qué caso un suavizado?

6. Retos (opcionales, suman puntos)

1. **Grabar a la velocidad real.** En el paso 1, crea el `VideoWriter` con los **FPS medidos** en lugar de 30 y comprueba que ahora el archivo dura lo mismo que la grabación.

2. **Convolución con ciclos `for`.** Programa la convolución 3×3 recorriendo píxel por píxel con dos `for` sobre un frame reducido a 160×90. Compara el resultado y el tiempo contra `cv2.filter2D`. ¿Cuántas veces más lento es?
3. **Mediana a mano.** Con `np.lib.stride_tricks.sliding_window_view`, calcula la mediana 3×3 de un frame en gris con NumPy y compárala con `cv2.medianBlur`.
4. **Histograma en color.** Dibuja en vivo los histogramas de los canales B, G y R de la webcam, cada uno con su color, y observa cómo cambian al acercar objetos de colores.

7. Errores frecuentes

Síntoma	Causa	Solución
`No se pudo abrir la fuente de video`	La cámara está ocupada (Zoom, Teams, la app Cámara) o no existe	Ciérrala, prueba `cv2.VideoCapture(1)` o usa `--video`
La ventana se congela o no aparece	Falta `cv2.waitKey` en el ciclo	Un `waitKey` por vuelta del ciclo
`error: (-215) ... createTrackbar` o las barras no aparecen	La ventana no existía al crear la barra	`cv2.namedWindow(VENTANA)` antes de `barra(...)`
`medianBlur` / `GaussianBlur` falla con kernel par	Esos filtros necesitan un tamaño **impar**	`k = 2 * valor + 1`
El `.mp4` grabado está vacío o no abre	No se llamó a `release()`, o el tamaño no coincide con el frame	`release()` al terminar; tamaño `(ancho, alto)` del frame
La cámara tarda mucho en abrir (Windows)	El backend MSMF es lento	`cv2.VideoCapture(0, cv2.CAP_DSHOW)`
macOS: la cámara no abre	Falta el permiso de cámara para la Terminal o VS Code	*Configuración → Privacidad → Cámara*

8. Entregable

Sube **un** `.zip` con:

1. Los scripts `p2_paso1_video.py`, `p2_paso2_suavizado.py` y `p2_paso3_realce.py`, funcionando.
2. Un **reporte PDF** (máximo 4 páginas) con capturas hechas **con tu webcam** (no con el video sintético), la tabla de PSNR del paso 2 y las respuestas a las preguntas.

Criterio	Puntos
Los 3 scripts corren con webcam y con `--video` y muestran los resultados esperados	4
Código ordenado y comentado; cada paso parte de la misma `[BASE]`	2
Respuestas correctas y justificadas con mediciones (FPS, PSNR, ms)	3
Reporte claro, con capturas de tu cámara	1
Retos (extra)	hasta +1

Referencia rápida

Función	Qué hace
`cv2.VideoCapture(0)` / `cv2.VideoCapture("v.mp4")`	Abre la cámara 0 o un archivo
`captura.read()` → `(ok, frame)`	Lee el siguiente frame
`captura.get(prop)` / `captura.set(prop, v)`	Lee / pide una propiedad (`CAP_PROP_FPS`, `CAP_PROP_FRAME_WIDTH` …)
`cv2.VideoWriter(ruta, fourcc, fps, (ancho, alto))` + `.write(frame)` + `.release()`	Graba video
`cv2.namedWindow`, `cv2.createTrackbar`, `cv2.getTrackbarPos`	Ventana y barras deslizantes
`cv2.filter2D(img, -1, kernel)`	Convolución con un kernel propio
`cv2.blur`, `cv2.GaussianBlur`, `cv2.medianBlur`, `cv2.bilateralFilter`	Filtros de suavizado
`cv2.PSNR(a, b)`	Calidad de `b` respecto a `a` (dB)
`cv2.convertScaleAbs(img, alpha=a, beta=b)`	`a·img + b` saturado a 0..255
`cv2.equalizeHist(gris)`, `cv2.createCLAHE(clip, (8, 8)).apply(gris)`	Ecualización global y local
`cv2.addWeighted(a, wa, b, wb, 0)`	`wa·a + wb·b` (unsharp mask)
`cv2.calcHist([gris], [0], None, [256], [0, 256])`	Histograma